

CSI473 Project Design Report

Campus Concierge - Team 15

Field	Complete
Assignment/phase	Phase 2 report
Approved project title	Campus Concierge
Approval decision/date	Approved with minor conditions
Team number	Group 15
Members and IDs	Thebe Segootsane 202306366; Kaelo Wadingalo 202306842; Hope Katso Toki 202307521; Katlo Ronald Benson 202304358
Repository URL	https://github.com/kaelowadingalo1-sys/CampusConcierge-
Submission tag	phase2-submission
Date	30 October 2026

This report extends the Phase 1 baseline (tag phase1-baseline-v1, 11 September 2026). Sections 1 to 7 are the Phase 1 baseline, updated where Phase 2 work corrected them (listed in Section 14.3). Sections 8.2 to 16, the References and the Appendices are new in Phase 2.

1. Executive summary

Campus Concierge is a peer-to-peer campus errand platform for the University of Botswana. A requester posts an errand under an approved category; the system automatically matches it to an available, vetted helper; and both parties keep a traceable, timestamped record of the agreed terms (task, category, standard price and deadline). The approved problem, evidence and scope are set out in Section 2.

The first vertical slice is limited to two low-risk categories (Parcel Collection and Non-assessed Document Printing/Collection), based on six lecturer conditions (R1-R6). These are addressed through D-001 to D-007, covering admin-managed categories, prohibited errands, 10-minute acceptance and automatic reassignment, cancellation/no-show/dispute rules, no payment-data handling, controlled prototype evaluation, and a Level 0 system context.

Sections 4-7 trace FR-01 to FR-17, five quality scenarios, the domain model and business rules, and the errand lifecycle. The Section 6 traceability matrix links requirements to use cases, analysis elements and verification evidence.

Section 8 records the architecture drivers and the Phase 2 decision: a single modular application with one PostgreSQL database and ports to the identity and notification services (ADR-001). Sections 9 and 10 give the data, API, deployment and failure design and the detailed design: ports and adapters, repositories inside a unit of work, and a transactional outbox, built in TypeScript on Node.js and PostgreSQL (ADR-002), with a helper-selection strategy and a message broker explicitly rejected. Section 11 designs the requester and helper tasks, with alternative states and an accessibility checklist.

Sections 12 to 14 describe the working vertical slice (Post Errand Request through helper acceptance, requester confirmation and the contact-detail gate), its 21 automated tests, and an honest reconciliation of design, code and tests: requirements FR-01 to FR-05 and FR-13 are implemented at API level, while cancellation, no-show, dispute, administrator functions and a user interface are not built. Section 15 records technical debt and risks. The recommendation is a conditional release as a supervised demonstration with synthetic data only; the system is not ready for production or real student use.

2. Problem evidence, stakeholders and scope

2.1 Problem statement

Students at the University of Botswana regularly need small errands run: collecting a parcel from a courier point, standing in a long registration or bank queue, buying an item in town, or printing and submitting a document before a deadline. Lectures, labs and part-time commitments often make it impossible for a student to do this themselves in time. Students currently ask around informally, usually in residence WhatsApp groups or by word of mouth, and this works unevenly: a request can sit unanswered for hours, a willing helper is not always someone the requester knows or trusts, and once someone agrees there is no shared record of what was actually agreed, including the task, the price and the deadline. When something goes wrong, such as a helper who does not show up, buys the wrong item, or a disagreement over whether payment was made, neither side has anything to point back to. Campus Concierge addresses this by giving students one shared place to post an errand, get automatically matched with a vetted, registered helper, and keep a traceable record of the deal as it happens.

2.2 Evidence of the problem

Evidence/source	Date/origin	What it shows
Informal survey of 12 students across two residence WhatsApp groups	Aug 2026	9 of 12 respondents said they had asked around for help with an errand at least once this semester; several reported requests going unanswered for over an hour
Short interviews with 4 students who had informally helped with or requested an errand	Aug 2026	All 4 said there was no shared record of what was agreed; 2 described a disagreement afterward over price or timing

2.3 Approval conditions addressed

The proposal was approved with minor conditions (six conditions, R1-R6). Each is addressed by a design record in the repository:

Ref	Lecturer's condition	Addressed by
R1	Restrict the first vertical slice to one or two low-risk campus errand categories.	D-001
R2	Prohibit errands involving bank/identity representation, assessed-work submission, confidential documents, restricted goods, high-value purchases, unsafe off-campus travel, or impersonating the requester.	D-002
R3	Define helper acceptance timeouts, reassignment, cancellation, requester no-show, helper no-show, and dispute-resolution rules.	D-003, D-004
R4	Do not process or store payment information.	D-005

Ref	Lecturer's condition	Addressed by
R5	Evaluate matching using controlled prototype scenarios; do not run a real pilot unless formal permission is obtained.	D-006, prototype/scenarios/, tests/matching-scenario-results.md
R6	Submit a Level 0 system context diagram showing requester, helper, administrator, and relevant notification/identity services.	models/context/level-0-context.drawio (Figure 2.1)

2.4 Level 0 system context

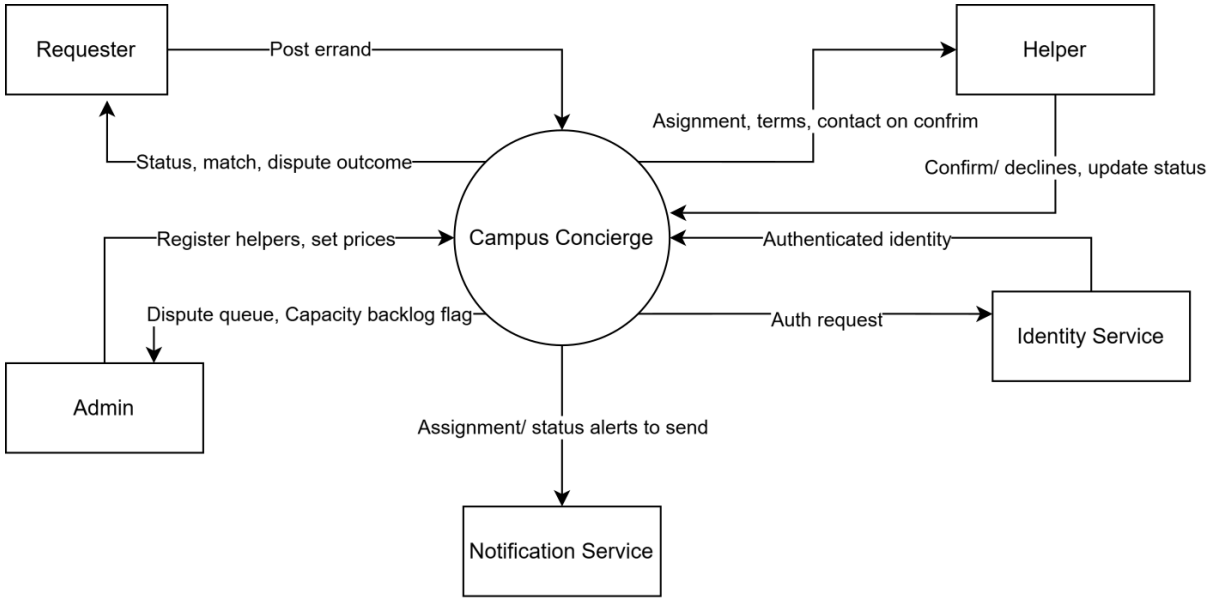


Figure 2.1: Level 0 system context diagram (models/context/level-0-context.drawio).

2.5 Goal and objectives

Goal: enable students to find a trustworthy peer helper for a campus errand and keep a shared, traceable record of the agreed deal, so disputes and unresolved no-shows become rare rather than routine.

- Objective 1: in controlled prototype test scenarios, correctly match at least 80% of scenario cases with an eligible helper within the 10-minute acceptance window, without a real pilot (per D-006).
- Objective 2: ensure at least 90% of matched deals have an explicit, timestamped record of task category, standard price and deadline stored before the helper marks work as started.
- Objective 3: provide a dispute-report workflow that captures enough evidence to reconstruct at least 90% of test dispute scenarios without relying on external chat logs.

2.6 Stakeholders

Stakeholder/role	Need	Concern	Evidence/access
Requester (student needing an errand run)	Find a willing helper quickly and trust the deal will be honoured	Helper doesn't show, or terms are disputed after the fact	Team members are themselves potential requesters; accessible via peer interviews
Helper (vetted student registered to run errands)	Get auto-assigned errands and clear task terms	Unfairly blamed with no record of what was actually agreed	Accessible via peer interviews with students who have informally helped others
App admin (helper vetting and assignment)	A reliable pool of helpers, and standard prices per category that stay fair	Not enough available vetted helpers, or a category price that drifts out of step with real effort	Team's own operational role running the vetting and registration process
Residence/hall administration	Deliveries and meetups don't disrupt hall or accommodation operations	Unknown visitors arriving at hall entrances for handoffs	Informal conversation with a hall porter about visitor handoffs at the entrance
University IT	A real identity system would need student accounts to be genuinely trustworthy	The prototype's mocked login (D-007) does not prove real integration with UB systems would work	No direct engagement this semester (D-007); UB's Microsoft 365 use confirmed only via UB's public site

2.7 Scope

In scope

- Posting an errand request under a defined category with its standard price

- Registering students as helpers, and auto-assigning a posted errand to an available registered helper
- Recording task details, category and deadline against the category's standard price
- Errand status lifecycle and dispute reporting

Out of scope

- Real payment processing or holding funds
- Per-errand price negotiation between requester and helper
- Open self-signup as a helper without admin approval
- University disciplinary adjudication
- Background or criminal-record checks beyond the team's own vetting

2.8 Assumptions and constraints

Assumptions

- Students authenticate with their existing university student number/email
- Cash or mobile money changes hands outside the app
- New helpers are recruited and approved by the admin team, not by open self-signup
- Each errand category has a fixed standard price set by admins, not negotiated per errand

Constraints

- Must remain usable on limited campus data or wifi
- Must be deployable and supportable by a small student team
- Errand capacity is limited by how many vetted helpers are currently registered
- Delivered within one semester as a vertical slice, not a full product

3. Glossary

Terms used consistently across the problem statement, stakeholder table and all downstream artefacts.

Term	Working definition
Requester	A student who posts an errand they need completed.
Helper	A student who has been vetted and registered by an administrator to complete errands.
Errand	A request in a defined category with task details, a deadline and a standard price.
Errand category	An administrator-managed grouping of errands that has one fixed standard price.
Deal	The traceable agreement created when a helper confirms an assigned errand.
Standard price	The non-negotiable price recorded for an errand category; the system records it but does not process payment.
Assignment	The system's allocation of a posted errand to the next available registered helper.
Reassignment	Allocation to another eligible helper after a decline or missed response window.
Backlog	The state of a posted errand with no eligible helper currently available; retried as helpers free up.
Dispute	A requester or helper report that an errand was not completed as agreed, supported by the recorded deal.
App admin	The role that vets helpers, maintains category prices and monitors assignment capacity.
Identity Service	The external entity a real login would depend on; mocked for this semester's prototype (D-007).
Notification Service	The external channel used for signup confirmation and assignment/status/dispute alerts.
Errand status	The lifecycle a single errand moves through: POSTED, AWAITING_ACCEPTANCE, ACCEPTED, IN_PROGRESS, COMPLETED, CLOSED, with exception paths to REASSIGN, CANCELLED or DISPUTED (Figure 7.1).

4. Functional requirements and use cases

4.1 Actors

- Requester - a student who posts an errand
- Helper - a vetted, registered student who is assigned errands
- Admin - vets helpers, sets category prices, resolves disputes
- Notification Service (supporting, external) - delivers assignment/status/dispute alerts
- Identity Service (supporting, external) - authenticates users; mocked for this semester's prototype (D-007)

4.2 Use-case diagram

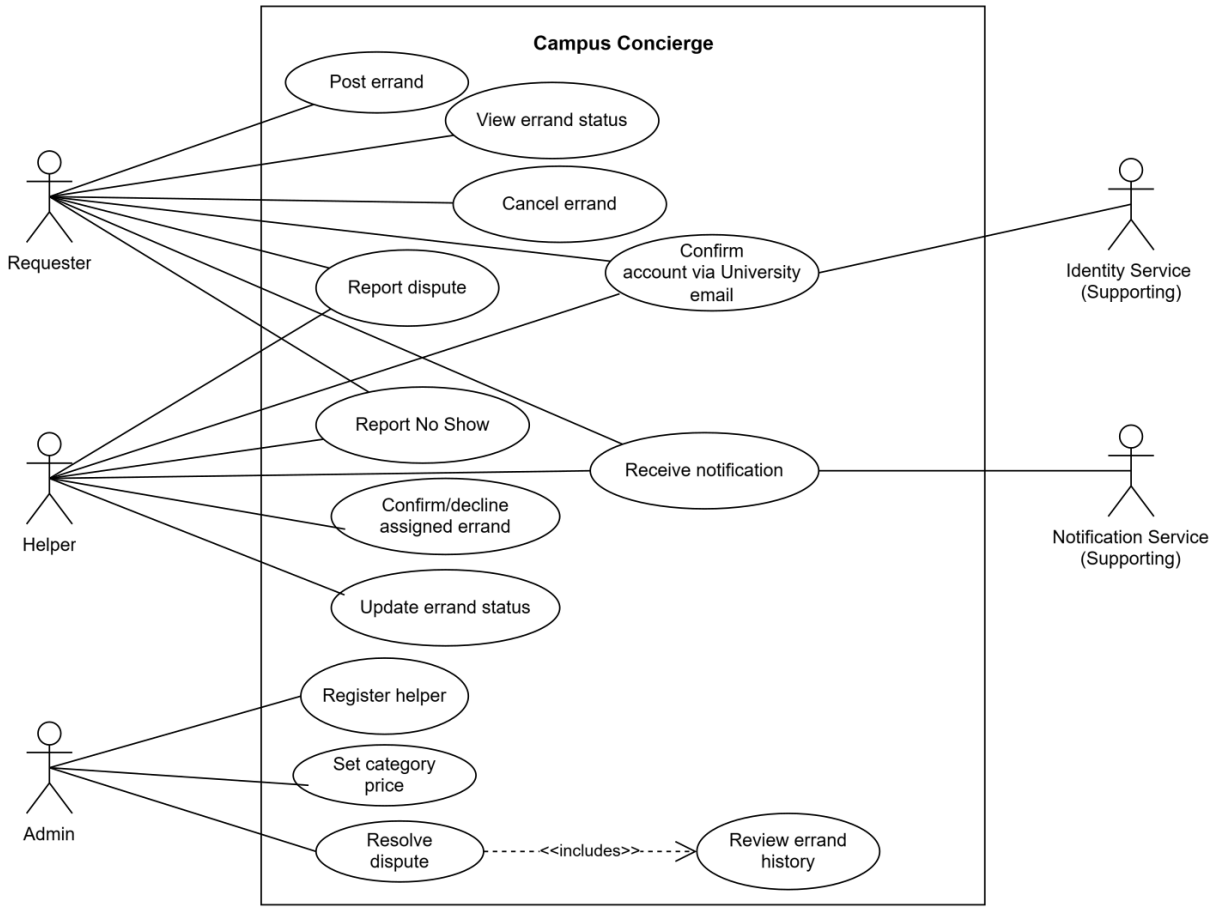


Figure 4.1: Campus Concierge use-case diagram (models/use-cases/campus-concierge-use-cases.drawio).

4.3 Functional requirements

Each requirement is written as an actor goal and traces to the design-record decision that justifies it. This list covers the first vertical slice only (Parcel Collection and Non-assessed Document Printing/Collection, per D-001).

ID	Requirement	Traces to
FR-01	A requester can post an errand request under an approved category, with task details and a deadline.	D-001, D-002
FR-02	The system automatically assigns a posted errand to an available registered helper.	D-003
FR-03	An assigned helper can confirm or decline the errand within the acceptance window.	D-003
FR-04	If a helper declines or does not respond within the acceptance window, the system reassigns the errand to the next eligible helper.	D-003
FR-05	Once an errand is confirmed, the system locks the agreed terms (task, category, standard price, deadline) and reveals contact/meeting details to both parties.	D-003, D-005
FR-06	A requester can cancel an errand freely before acceptance; cancellation after acceptance requires a stated reason and is recorded.	D-004
FR-07	A helper can report a requester no-show after waiting the defined window at the handoff point.	D-004
FR-08	A requester can report a helper no-show if the helper does not appear within the agreed window.	D-004
FR-09	Either party can open a dispute on a contested or failed errand; the system captures the full recorded event history as evidence.	D-004
FR-10	An administrator can review a dispute's recorded evidence and resolve it with a retained, timestamped outcome.	D-004
FR-11	An administrator can vet and register a student as a helper.	D-002 (App admin stakeholder)
FR-12	An administrator can set and adjust the standard price for an errand category.	Category pricing decision

ID	Requirement	Traces to
FR-13	The system rejects an errand posted under a prohibited or non-approved category at creation, not later.	D-001, D-002 (BR-01, BR-02)
FR-14	The system never requires a helper to perform financial, identity, academic-assessment, or impersonation representation, regardless of category.	D-002 (BR-03)
FR-15	The system does not process, transfer, or store payment information; only the category's standard price is recorded as data.	D-005
FR-16	A student creates an account by confirming a link sent to their university email address; the account is self-service after that.	D-007
FR-17	A requester or helper receives status, assignment, and dispute notifications through the Notification Service.	Level 0 context diagram

4.4 Detailed use case: Post Errand Request

This is the use case worked through in full detail; it anchors the acceptance criteria (4.5) and the interaction walkthrough in Section 7.

Actor: Requester

Precondition: The requester has a confirmed account (D-007).

Postcondition: A validated errand exists in POSTED status, tied to an approved category and its current standard price, with the requester able to see its status.

Main success scenario

- 1. Requester selects an approved errand category.
- 2. Campus Concierge retrieves the category's current standard price and general backlog status.
- 3. Requester supplies the task details and a deadline.
- 4. Campus Concierge validates the category against the approved/prohibited list (BR-01, BR-02).
- 5. Campus Concierge records the errand as POSTED, with the category's standard price locked in as the recorded price.
- 6. Campus Concierge confirms the errand identifier and status to the requester.

Alternative flows

Scenario	Required system response	Design principle
Category is not on the approved list	Reject the request before creating any errand record.	Access policy (BR-01, BR-02 / D-001, D-002)
No eligible helper is currently available	Still record the errand as POSTED; flag the capacity backlog to the Administrator.	Failure policy (D-003)
Requester's account is not yet confirmed	Refuse posting until university-email confirmation completes (D-007).	Access policy
Duplicate submission (same task resubmitted quickly)	Return the existing pending errand rather than creating a second one.	Idempotency
Category's standard price changed between the requester viewing it and submitting	Use the current standard price at the moment of submission, not a cached one from step 2.	Version identity

4.5 Acceptance criteria

ID	Given	When	Then
AC-1	An approved category, complete task details and a deadline	The requester submits the errand	It is recorded as POSTED with the category's current standard price locked in
AC-2	A category not on the approved list (D-002)	The requester attempts to submit	The system rejects it before any errand record is created
AC-3	No eligible registered helper is currently available	An errand is posted	It stays POSTED, enters the backlog, and a capacity flag is raised to the administrator
AC-4	An assigned helper has not responded	10 minutes elapse (D-003)	The errand reassigns to the next eligible helper and the timeout is recorded against the original helper
AC-5	The requester's account is not yet confirmed via university email (D-007)	They attempt to post an errand	The system refuses posting until confirmation completes

5. Quality requirements

Five quality scenarios, each a distinct quality attribute, written in stimulus/context/response/measure form, and each traced to the design record and (where one exists yet) the controlled prototype scenario that evaluates it.

#	Attribute	Stimulus	Context	Response	Measure
QS-1	Reliability	Assigned helper does not accept or decline	Helper has one active assignment; device may be offline or silent	Assignment expires; errand reassigns to the next eligible helper; timeout recorded against the original helper (D-003)	Acceptance window fixed at 10 minutes from assignment creation; expiry detected and next assignment created within 1 minute of expiry; no errand in AWAITING_ACCEPTANCE for more than 11 minutes from the original assignment
QS-2	Privacy	An errand is posted and awaiting a match	Other students can browse open errands	Contact and meeting details stay hidden until a helper's assignment is confirmed by both parties	Zero contact-detail exposure to any party before confirmation, checked on every posted errand
QS-3	Availability	An errand is posted with zero eligible helpers currently free	Normal operation, small registered helper pool	Errand enters a backlog state; a capacity flag is raised to the administrator; assignment retries as helpers free up	Backlog flag raised within the same request cycle that found zero helpers; no errand silently dropped
QS-4	Auditability	A dispute is opened on a contested errand	Either party can open a dispute at any point after acceptance	The system reconstructs the full event history (assignment, confirmation, status changes, timestamps) for admin review	100% of recorded fields available to the admin; no dispute reviewed with missing event history
QS-5	Scalability	Several errands are posted at close to the same time	Helper pool smaller than the number of posted errands	Errands are assigned in a defined order (e.g. post-time); the remainder enter backlog; no helper receives two active assignments	Zero double assignments across a batch of up to 50 errands posted within 10 seconds against 20 helpers; every errand beyond the 20th enters backlog in the same request cycle

5.1 Resulting design obligations

Each scenario places a concrete obligation on the design, independent of which architecture eventually satisfies it:

- QS-1 -> D-003; evaluated by prototype/scenarios/helper-timeout.md. Obligation: the system must be able to expire an assignment and reassign without manual intervention.
- QS-2 -> D-004, D-007. Obligation: contact/meeting detail must not be readable through any path before both-party confirmation.
- QS-3 -> D-003; evaluated by prototype/scenarios/no-helper-available.md. Obligation: 'no eligible helper' must be a first-class, recorded state, not a silently dropped request.
- QS-4 -> D-004, D-007. Obligation: every status change, assignment attempt and timestamp on an errand must be retained and retrievable as a single, complete history.
- QS-5 -> evaluated by prototype/scenarios/multiple-errands-compete.md. Obligation: assignment must be atomic enough that two concurrent posts can never both capture the same helper.

QS-2 and QS-4 do not yet have a dedicated controlled prototype scenario written (see docs/quality-scenarios.md); this is flagged again in the traceability matrix, Section 6.4.

6. Analysis/domain model and responsibilities

6.1 Domain model

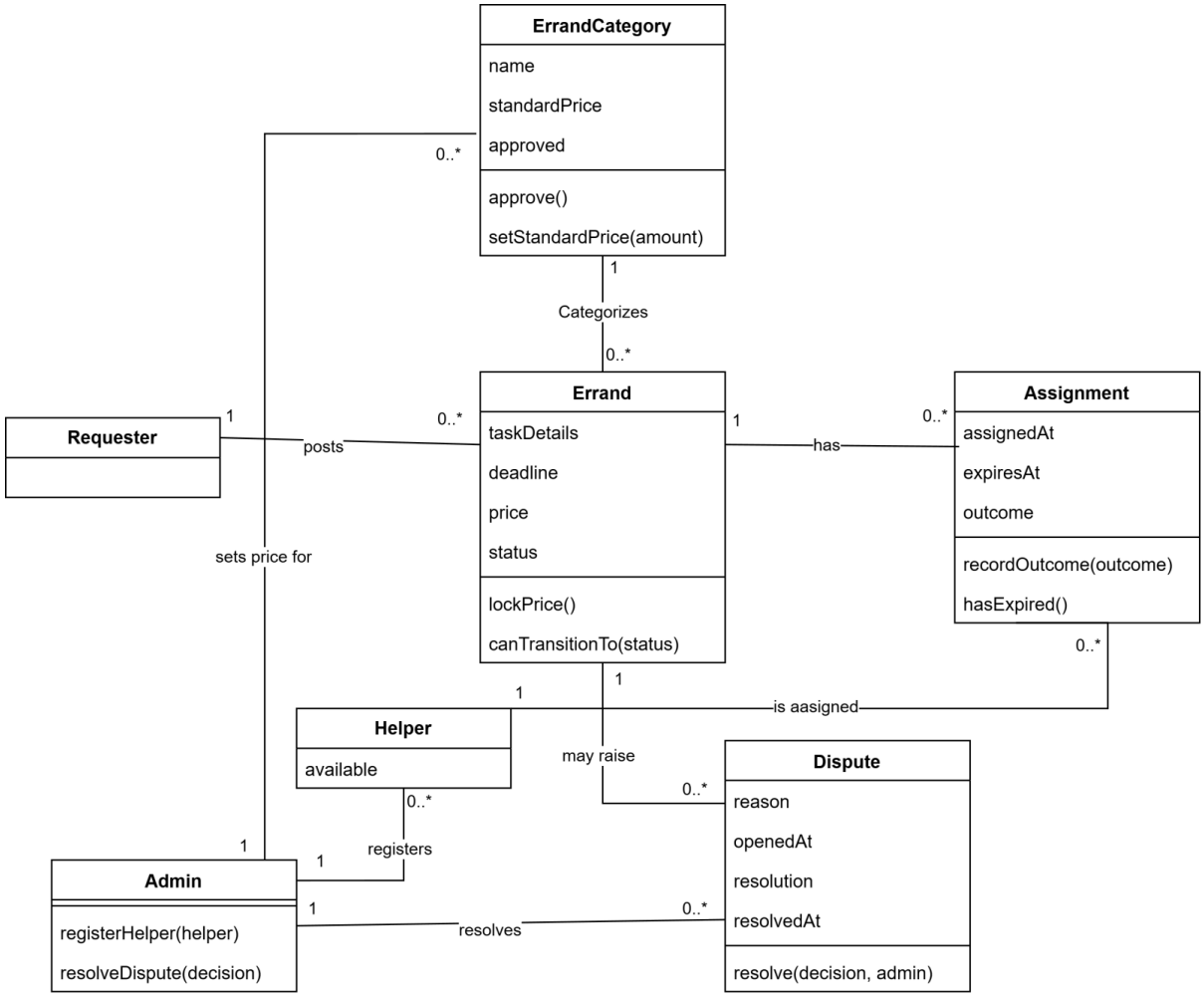


Figure 6.1: Campus Concierge domain class model (models/domain-model.drawio).

Two scope decisions were made while building this model (D-008, status: proposed):

- Requester, Helper and Administrator are separate classes, not roles of one User class, because a single User-with-role class would carry Helper-only state (available) and Administrator-only behaviour (setting prices, resolving disputes) as optional fields most users never populate.
- Assignment is its own persistent class, not a field on Errand, because an errand can accumulate a history of assignment attempts (decline, timeout, reassignment, per D-003), which a single currentHelperId field cannot represent, and D-004 needs that full history as dispute evidence.

6.2 Business rules

Domain invariants (enforced by the domain model itself)

- BR-D1: An Errand's category must be approved at the moment the Errand is created (D-002, BR-01/BR-02).
- BR-D2: An Errand's price is fixed to its category's standard price at post time and does not change even if the category's price changes later (D-001, pricing decision).
- BR-D3: A Helper can be the subject of at most one active (non-terminal) Assignment at a time (D-003, no double-assignment).
- BR-D4: An Assignment's outcome, once recorded, is immutable (supports audit and dispute evidence, D-004).
- BR-D5: An Errand may have at most one Dispute, since DISPUTED is a terminal lifecycle state (D-004).
- BR-D6: Only an Administrator may change an ErrandCategory's approved flag or standard price (D-002, pricing decision).
- BR-D7: A Dispute's resolution can only be set by an Administrator, never directly by the Requester or Helper (D-004).

Categories (D-001, D-002)

- Only two errand categories are enabled in the first vertical slice: Parcel Collection and Non-assessed Document Printing/Collection.
- Prohibited categories, at any stage: bank/identity representation, submission of assessed academic work, confidential/sensitive documents, restricted goods, high-value purchases, unsafe off-campus travel, or anything requiring the helper to impersonate the requester.
- BR-01: only enabled, admin-approved categories may be posted.

- BR-02: a prohibited-category errand is rejected at creation, not caught later.
- BR-03: a helper is never asked for financial, identity, academic-assessment, or impersonation representation.
- BR-04: the first slice's UI and API only expose the two approved categories.

Assignment and timeouts (D-003)

- A helper has 10 minutes to accept or decline an assigned errand.
- Decline or timeout leads to immediate reassignment to the next eligible helper.
- Every declined/expired assignment is recorded against the original helper.
- No eligible helper available leads to the errand entering a backlog state and retrying as helpers free up.

Cancellation and no-shows (D-004)

- Requester can cancel freely before a helper accepts; after acceptance, cancellation needs a stated reason and is recorded.
- Helper waits 10 minutes at the handoff point before reporting a requester no-show.
- Requester reports a helper no-show if the helper doesn't appear in the agreed window; either no-show can be escalated to a dispute.

Disputes (D-004)

- Either party can open a dispute on a contested or failed errand; the system captures the full recorded event history as evidence.
- An app admin reviews and resolves the dispute; the resolution is final within this system (no automated or legal adjudication).

Payment (D-005)

- No payment is processed, transferred, or held by the system, and no payment credentials are ever stored.
- Only the category's fixed standard price is recorded as data, as part of the deal.

Evaluation and identity (D-006, D-007)

- The matching/assignment logic is evaluated against controlled, synthetic scenarios; no real pilot runs without separate formal permission.
- The identity service is mocked for the prototype rather than integrated with a real system, since D-006 already rules out a real pilot.

6.3 CRC cards

Class	Responsibilities	Collaborators
Errand	Know its own task details, category, price, deadline and status Validate that a requested status transition is legal Lock its price to the category's standard price at posting time (BR-D2)	ErrandCategory, Assignment, Dispute, Requester
Assignment	Know which helper it was offered to, and when Record its own outcome once decided: accepted, declined, or timed out Know whether its acceptance window has expired	Errand, Helper
ErrandCategory	Know its own approved status and standard price Refuse to let a new Errand reference it while not approved (BR-D1)	Errand, Administrator
Administrator	Vet and register a Helper Set an ErrandCategory's approved flag and standard price Review a Dispute's evidence and resolve it	Helper, ErrandCategory, Dispute
Dispute	Know its reason, the Errand it concerns, and when it was opened Record its resolution and when it was resolved Refuse a resolution set by anyone other than an Administrator (BR-D7)	Errand, Administrator

6.4 Traceability matrix

Requirement -> use case -> analysis element -> verification.

Requirement	Use case	Analysis element	Verification
FR-01	Post Errand Request	Errand, ErrandCategory	AC-1, AC-2
FR-02	Post Errand Request	Assignment	prototype/scenarios/multiple-helpers-available.md, AC-3
FR-03	Decline Errand Assignment	Assignment	prototype/scenarios/helper-accepts.md, helper-declines.md

Requirement	Use case	Analysis element	Verification
FR-04	Decline Errand Assignment	Assignment	AC-4, prototype/scenarios/helper-declines.md, helper-timeout.md, QS-1
FR-05	Post Errand Request (confirmation phase)	Errand, Assignment	AC-1, QS-2
FR-06	Cancel Errand	Errand	AC-6: cancel from POSTED or AWAITING_ACCEPTANCE is free and ends in CANCELLED; cancel after ACCEPTED needs a reason
FR-07	Report No Show	Dispute	AC-7: helper reports after waiting the defined window; one Dispute is created
FR-08	Report No Show	Dispute	AC-8: requester reports a helper who has not appeared in the agreed window
FR-09	Report Dispute	Dispute, Assignment, Errand	QS-4, AC-9: a second Dispute on the same errand is refused (BR-D5)
FR-10	Resolve Dispute	Administrator, Dispute	QS-4, AC-10: a resolution set by a non-administrator is refused (BR-D7)
FR-11	Register Helper	Administrator, Helper	AC-11: only a registered helper receives assignment offers
FR-12	Set Category Price	Administrator, ErrandCategory	AC-12: a price change applies to new errands only (BR-D2, BR-D6)
FR-13	Post Errand Request	ErrandCategory, Errand	AC-2
FR-14	(policy, no single use case)	ErrandCategory	Review check: no approved category allows financial, identity, assessment or impersonation tasks (BR-03)
FR-15	(policy, no single use case)	Errand, price attribute only	Review check: no payment field exists in the domain model, only the locked category price
FR-16	Confirm Account via University Email	outside domain model, see D-007	AC-5
FR-17	Receive Notification	outside domain model, external service	AC-17: offer, helper-accepted, contact-reveal and rejection notifications are each sent once

The traceability matrix was updated in Phase 2: rows that were pending now name an acceptance criterion or a review check. AC-6 to AC-12 and AC-17 are defined in this matrix, but their full wording is not yet in docs/acceptance-criteria.md.

7. Interaction and lifecycle

7.1 Errand lifecycle (state model)

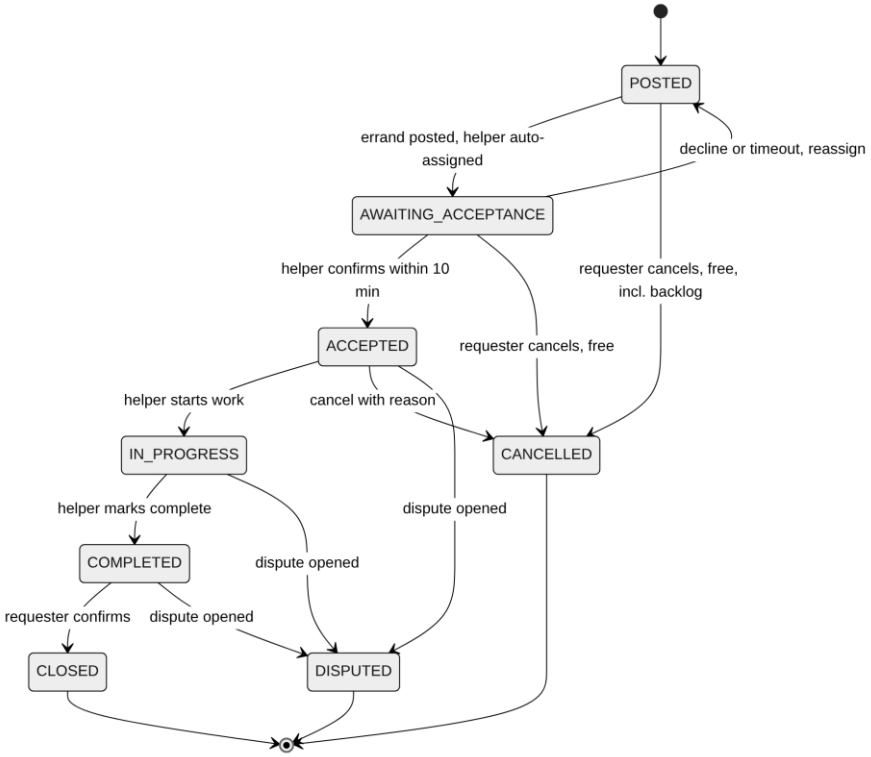


Figure 7.1: Errand lifecycle state diagram, revised in Phase 2 (source: models/state/errand-lifecycle.drawio).

An Errand starts at POSTED. Once a helper is auto-assigned it moves to AWAITING_ACCEPTANCE; the helper's confirmation within the 10-minute window (D-003) moves it to ACCEPTED, while a decline or timeout returns it to POSTED for reassignment to the next eligible helper. An errand with no eligible helper stays POSTED and is flagged as backlog (it is not a separate state). From ACCEPTED the helper starts work (IN_PROGRESS) and later marks it complete (COMPLETED); the requester's confirmation closes it (terminal CLOSED). CANCELLED is terminal and reachable from POSTED (including backlogged errands) or AWAITING_ACCEPTANCE (free cancellation), or from ACCEPTED (cancellation with a stated reason, D-004). DISPUTED is terminal and reachable from ACCEPTED, IN_PROGRESS or COMPLETED, reflecting that either party may open a dispute at any point after acceptance (D-004, QS-4).

7.2 Interaction: Post Errand Request

Figure 7.2 formalises the main flow. The requester posts the errand; Errand asks ErrandCategory whether the category is approved and what its standard price is, and rejects an unapproved category with a notification (opt frame, BR-D1); Errand locks the price (BR-D2). The first alt frame is the assignment decision: an eligible helper is found and offered the errand, or none is found and the errand enters backlog and the administrator is flagged (QS-3). The second alt frame is the helper's answer: the helper confirms within the window, the requester then confirms, and only when both have confirmed are the terms locked and the contact details released to both parties (AD-05, QS-2); or the helper declines or the window expires, which records the outcome on the Assignment and requests reassignment to the next eligible helper (AD-02).

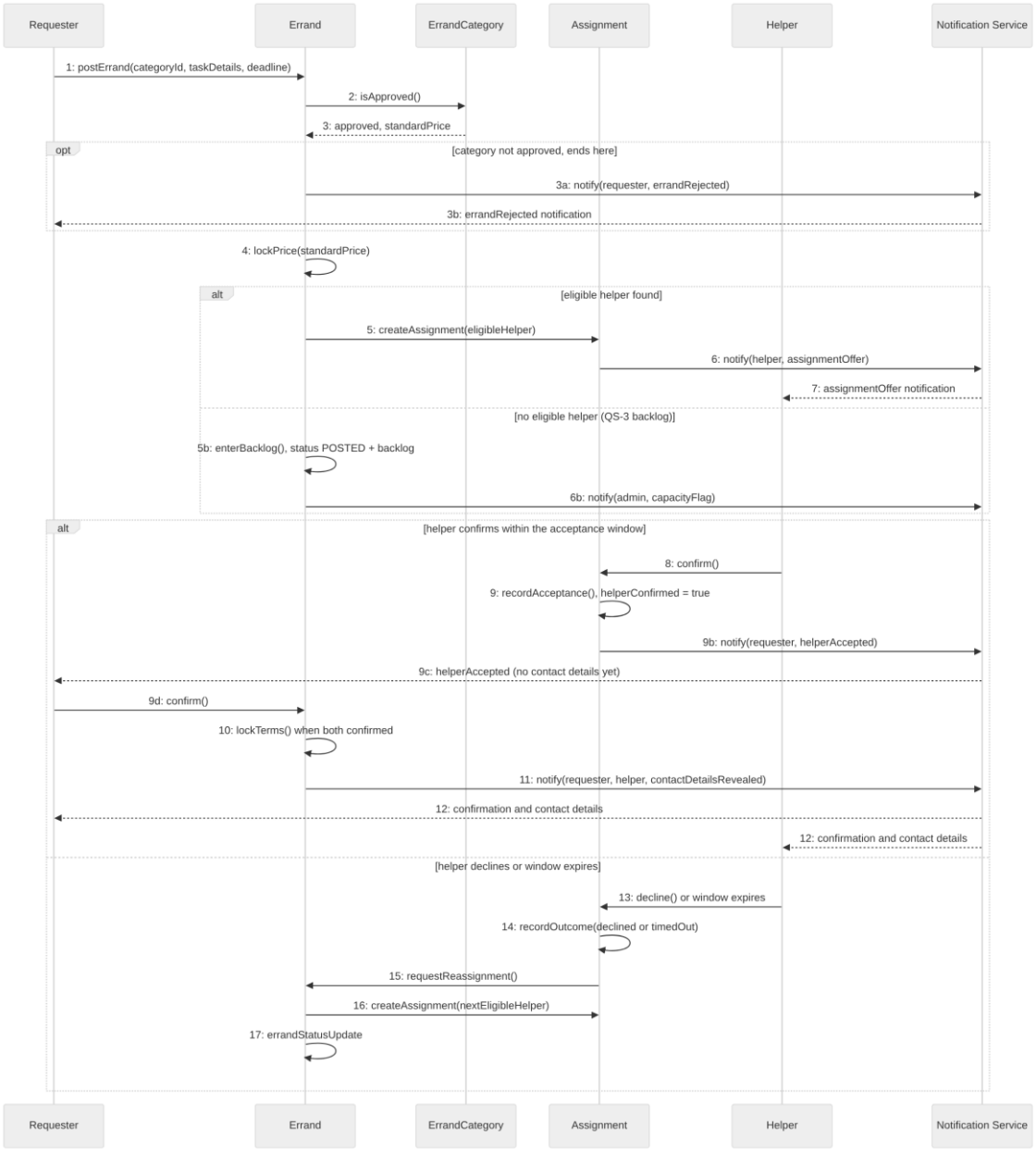


Figure 7.2: Sequence diagram for Post Errand Request, revised in Phase 2: rejection, backlog, requester confirmation and decline/timeout paths (source: models/sequence/sequence-core-use-case.drawio).

7.3 Consistency across models

The status values named in the domain model's Errand.status attribute (Section 6.1) are exactly the states drawn in Figure 7.1: POSTED, AWAITING_ACCEPTANCE, ACCEPTED, IN_PROGRESS, COMPLETED, CLOSED, CANCELLED and DISPUTED. Errand.canTransitionTo(status) is the operation responsible for enforcing that only the transitions drawn in Figure 7.1 are legal. The Assignment class's assignedAt, expiresAt and outcome attributes are what the AWAITING_ACCEPTANCE state and its decline/timeout transition are backed by, and Assignment.hasExpired() is the operation the QS-1 reassignment obligation (Section 5.1) depends on. Figure 7.2's messages 4, 10 and 14 (lockPrice, lockTerms, recordOutcome) are exactly those operations; message 14's declined/timedOut outcome and message 15's requestReassignment() are what drive the AWAITING_ACCEPTANCE -> POSTED transition back in Figure 7.1.

8. Architecture and decisions

Section 8.1 records the architecture drivers identified in the Phase 1 review. Section 8.2 records the Phase 2 work built on them: the quality-to-architecture obligations, the alternatives compared, the component model and the architecture decision record.

8.1 Architecture drivers identified during the Phase 1 review

Drivers were pulled from three sources, following the method used in class: functional responsibility, quality scenario, and constraint/dependency, then each was filtered by essence (is it central to the system?), meaning (what does it mean here?) and fit (what does it affect?).

ID	Source	Driver	Essence / meaning / fit
AD-01	Functional responsibility (FR-02 to FR-05)	Own the authoritative assignment of a posted errand to an eligible helper, including its full attempt history.	Essence: central, this is the system's reason to exist. Meaning: assignment state and history must be a first-class, queryable record, not a log line. Fit: affects Errand, Assignment, dispute evidence (QS-4), and any future matching logic.
AD-02	Quality scenario QS-1 (reliability)	A non-responding helper's assignment must expire and reassign within 10 minutes, unattended.	Essence: central, directly protects the requester's deadline. Meaning: expiry is a timed state transition, not a manual admin action. Fit: affects Assignment's lifecycle and whatever component owns time-based transitions.
AD-03	Quality scenario QS-5 (scalability under concurrency)	Two errands posted close together must never both capture the same helper.	Essence: central, a double-assignment breaks BR-D3 and the trust the whole system depends on. Meaning: assignment must be an atomic operation, not read-then-write. Fit: affects however Helper availability and Assignment creation are coordinated.
AD-04	Quality scenario QS-3 (availability with zero helpers)	A posted errand with no eligible helper must become a visible backlog entry, never a silently dropped request.	Essence: central to trust; a dropped errand is worse than a slow one. Meaning: 'no helper available' is a state, not an error path that terminates the request. Fit: affects Errand's state model and whatever notifies the Administrator of capacity backlog (context diagram, Figure 2.1).
AD-05	Quality scenario QS-2 (privacy)	Contact and meeting details must stay hidden until both parties have confirmed the assignment.	Essence: not central to the core workflow, but a hard safety/trust boundary. Meaning: this is an access-control rule that must hold regardless of which interface renders the errand. Fit: affects every read path that can expose an Errand's contact/meeting fields.
AD-06	Quality scenario QS-4 (auditability)	The full event history of an errand (assignment, confirmation, status changes, timestamps) must be reconstructable for a dispute review.	Essence: central, this is what makes the dispute workflow (FR-09, FR-10) possible at all. Meaning: history must be retained, not just current state. Fit: affects Assignment (BR-D4, immutable outcome) and however Errand's status history is stored.
AD-07	Constraint/dependency (D-005)	The system must never process, transfer, or store payment information; only a category's fixed standard price is recorded as data.	Essence: not central to matching, but a non-negotiable boundary set by the lecturer condition R4. Meaning: no design may introduce a payment-credential field or a payment-processing call, however convenient. Fit: affects ErrandCategory.standardPrice and rules out an entire category of external integration.
AD-08	Constraint/dependency (D-007, R6)	A real identity system is out of scope this semester; the Identity Service is mocked, but stays on the system boundary as a named external entity.	Essence: not central to this semester's evaluation (D-006), but shapes where authentication sits. Meaning: authentication must be swappable behind a stable boundary, since the mock is explicitly provisional. Fit: affects how Requester/Helper/Admin identity is established without coupling the rest of the system to the mock's implementation.

ID	Source	Driver	Essence / meaning / fit
AD-09	Constraint/dependency (D-001, D-002)	The errand category list is closed and admin-managed; every errand-creation path must validate against it, not just the UI.	Essence: central to safety (R1, R2); this is a lecturer-imposed guardrail, not a style choice. Meaning: category validation is policy that must be enforced wherever an Errand can be created, not decoration on one screen. Fit: affects ErrandCategory, BR-01/BR-02 enforcement, and any additional entry point added later (e.g. an API).
AD-10	Constraint/dependency (project scope)	The system must be built, run and supported by a small student team within one semester, evaluated only against controlled scenarios (D-006), not a live pilot.	Essence: not part of the problem domain, but bounds every other decision's cost. Meaning: favour the simplest structure that satisfies AD-01 to AD-09 over one that anticipates scale or users this project will not have. Fit: affects the choice of architecture style and deployment granularity to be made in Phase 2.

8.2 Alternatives, component model, ADRs and consequences

Five quality scenarios and the project constraints were translated into obligations on the structure. The three that most strongly shape the architecture are QS-5, QS-2 and QS-4.

8.2.1 Quality scenarios to architecture

Scenario	Obligation on the architecture	Architectural response	Element
QS-5 Scalability (primary)	Checking that a helper has no active assignment and creating the assignment must be one indivisible step	Create assignments inside one database transaction, backed by a rule that allows only one active assignment per helper	Errand Workflow, relational database
QS-2 Privacy (primary)	Exactly one place decides whether contact details may be released	Contact and meeting details are readable only through the confirmation gate, which checks both confirmations	Errand Workflow (confirmation gate)
QS-4 Auditability (primary)	A status change and its history record can never disagree, and history cannot be edited	History is written in the same transaction as the change and is append-only; Dispute reads it and never writes it	Errand Workflow, Dispute
QS-1 Reliability	Expiry must still happen if the application restarts or a timer is missed	Store an expiry time on each Assignment; a scheduler checks stored times at start-up and at a short interval	Errand Workflow (expiry scheduler)
QS-3 Availability	An errand with no eligible helper is never lost and the administrator is told	Set backlog and send the capacity notice in the same request; retry when a helper becomes free	Errand Workflow, Notifier port

8.2.2 Alternatives compared

Two realistic structures were compared on the same criteria. Neither is a technology name; each is a structure with defined responsibilities.

- **Alternative A: modular application.** One deployable application with four parts (Web/API layer, Errand Workflow, Category and Helper Administration, Dispute), one relational database, and ports to the two external services. Each module owns its own tables.
- **Alternative B: separate services with a message queue.** An Errand service, a Matching service, a Dispute service and a Notification worker, each with its own data store, communicating through a queue and request calls.

Criterion	Alternative A	Alternative B	Better
No double assignment (QS-5)	One transaction checks and creates the assignment; a database rule enforces one active assignment per helper	The one-active-assignment rule spans two services and needs extra coordination	A
Contact gate (QS-2)	One gate in one module; another module could read the fields by mistake	Contact data can sit in one service's store, a stronger boundary	B, slightly

Criterion	Alternative A	Alternative B	Better
History agrees with state (QS-4)	Change and history written in one transaction	History and state in different services; a dispute review joins records from two places	A
Expiry on time (QS-1)	Scheduler in the same process as the data; catches up from stored times	Separate expiry worker; more parts to keep running	Equal
Coupling and team size	Modules coupled through code interfaces in one process	Services coupled through messages and versioned contracts	A
Operational cost (small team, one semester)	One thing to build, run and debug	Several services, a queue and monitoring	A
Security boundary	One boundary around the whole application	Several internal boundaries	B, slightly

8.2.3 Component model

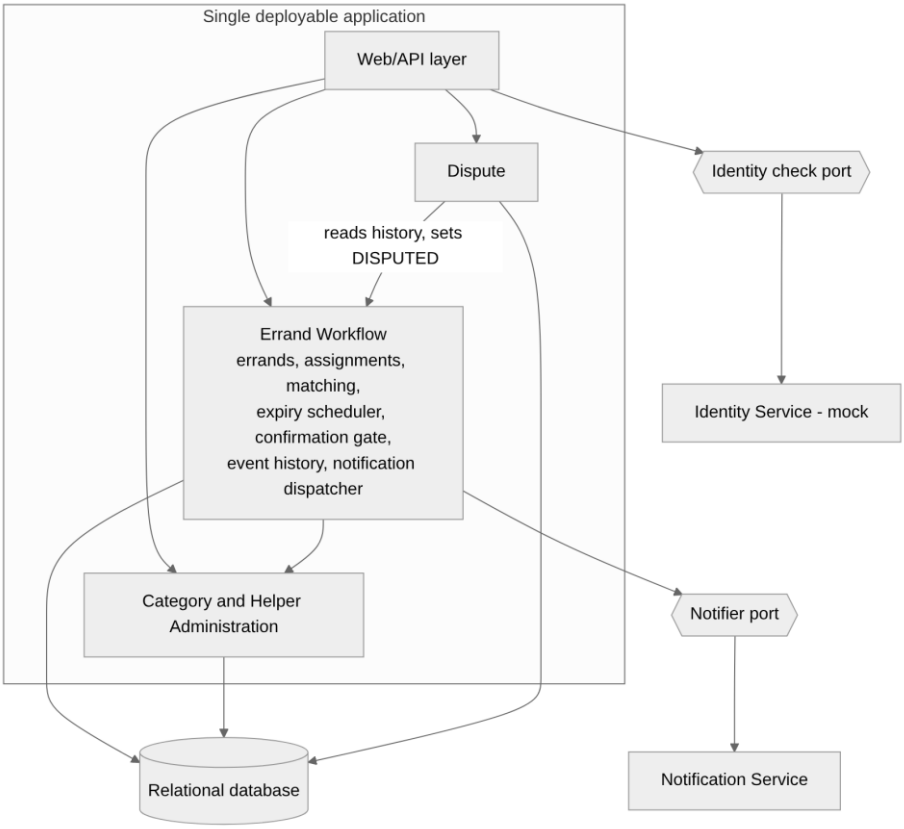


Figure 8.1: Component view of Alternative A (source: models/component-architecture.drawio; this figure is a rendering of the same model).

Module	Responsibility	Depends on
Web/API layer	Receives requests, checks identity, routes to modules	Identity check port, all modules
Errand Workflow	Errands, assignments, matching, expiry scheduler, confirmation gate, event history, notification dispatcher	Category and Helper Administration, Notifier port
Category and Helper Administration	Categories, standard prices, helper registry	none
Dispute	Disputes and administrator resolution	Errand Workflow (reads history, sets DISPUTED)

Dependencies run in one direction with no cycles.

8.2.4 Architecture decision (ADR-001) and consequences

Decision. Alternative A is chosen. It is better on the three criteria that matter most to this project's drivers (double assignment, history consistency, operational cost). Alternative B is slightly better on contact isolation and security

boundaries, which are mitigated by review and tests. The full record is decisions/ADR-001-architecture.md; Section 10 records ADR-002 (patterns and framework). The Phase 2 implementation follows this decision.

- **Positive:** double assignment is prevented by an ordinary transaction and a database rule; a status change and its history entry cannot disagree; one thing to build, run and debug; identity and notification can be replaced behind their ports.
- **Negative:** the whole system fails and scales as one unit; the contact-detail boundary is a module boundary, so it relies on review and a test; the expiry scheduler runs in the same process, so a stopped application delays reassignment until it restarts.
- **Highest risk:** expiry missed while the application is stopped (QS-1). Mitigation: the expiry time is stored on each Assignment and checked at start-up and at a 30-second interval.
- **We would reconsider if:** a concurrent test of 50 errands against 20 helpers produced any double assignment a database rule could not prevent; the contact-detail test failed repeatedly; missed expiries occurred despite stored-time catch-up; a real pilot is approved (D-006); or the team's capacity changes enough to carry separate services.

9. Data, API/service and deployment design

9.1 Logical data model

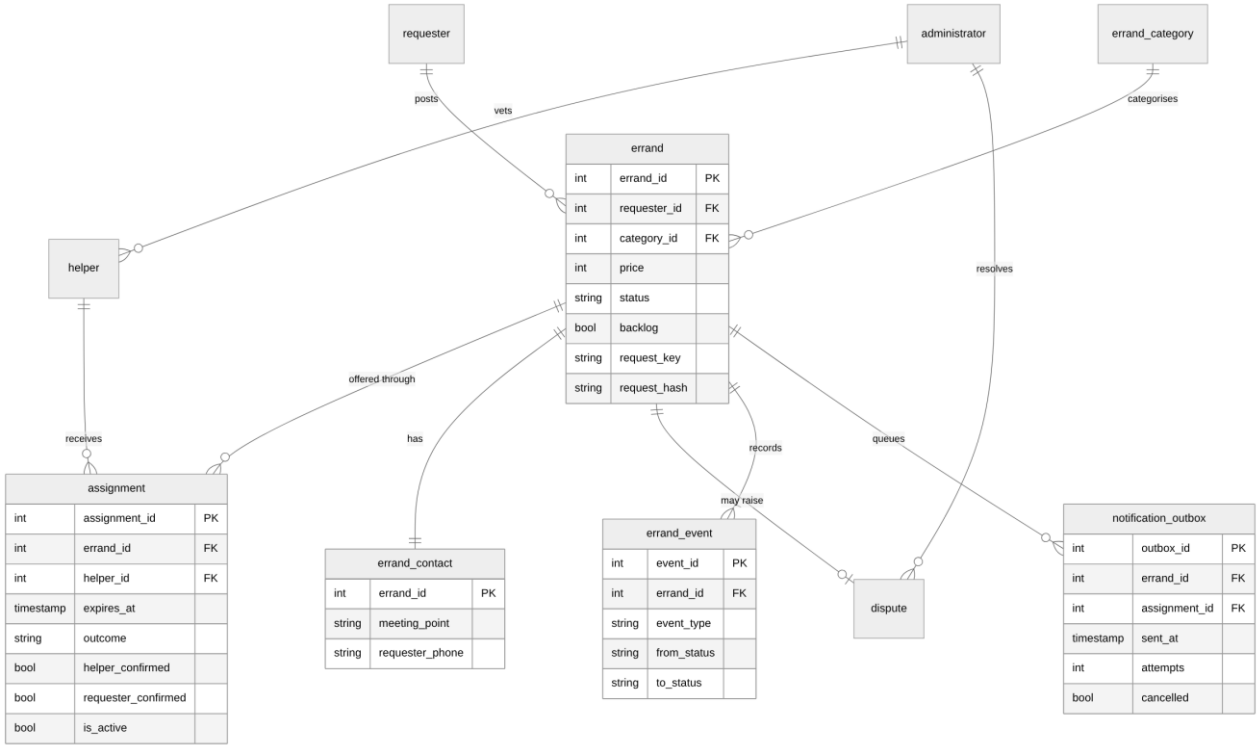


Figure 9.1: Logical data model, key columns only (source: models/logical-data-model.drawio; migrations-or-schema/001_init.sql).

The logical model keeps the domain classes (Requester, Helper, Administrator, ErrandCategory, Errand, Assignment, Dispute) and adds what the quality scenarios need: errand_contact (meeting point and requester phone, read only through the confirmation gate), errand_event (append-only history), notification_outbox (notifications sent after commit and retried), errand.request_key and request_hash (duplicate-submission handling) and assignment.is_active (so the database can enforce one active assignment). Backlog is a flag on a POSTED errand, matching the lifecycle.

ID	Integrity rule	Enforced by
DI-1	A helper has at most one active assignment (BR-D3, QS-5)	Unique rule on assignment(helper_id) where is_active; one transaction for check and create
DI-2	An errand has at most one active assignment	Unique rule on assignment(errand_id) where is_active
DI-3	An assignment outcome, once set, cannot change (BR-D4)	Database trigger
DI-4, DI-5	One dispute per errand; only an administrator sets a resolution (BR-D5, BR-D7)	Unique rule; check that resolved_by is set with resolution
DI-6	The errand price equals the category price at posting and never changes (BR-D2)	Price copied into errand.price
DI-7	The same posting key never creates a second errand	Unique rule on (requester_id, request_key); request_hash compared on reuse

ID	Integrity rule	Enforced by
DI-8	Contact details readable only after both parties confirm (QS-2)	Held in errand_contact; read only by the confirmation gate
DI-9, DI-10	Every status change has a history record in the same transaction; history is never edited or deleted (QS-4)	One transaction; append-only trigger
DI-11	An unsent notification belongs to a live assignment	Cancelled in the same transaction that closes the assignment
DI-12	A notification is delivered at most once per outbox row	Outbox id sent as the idempotency key

9.2 API contract: Post Errand Request

The core operation is POST /errands. It realises the Post Errand Request use case, FR-01, FR-02 and FR-13, and the acknowledgement in use-case step 6 (the response gives the errand identifier and status). The caller is identified by the identity check port, not by a field in the body. Full contract: docs/api-contracts/core-operation.md.

Part	Field	Rule
Header	Idempotency-Key	Required, 8 to 64 characters; identifies one user intent; stored as errand.request_key
Body	categoryId	Must exist and be approved at the moment of the request (BR-D1, BR-01, BR-02)
Body	taskDetails	1 to 500 characters, not blank
Body	deadline	Date and time later than the time of the request
Body	meetingPoint	1 to 200 characters; stored in errand_contact; never returned in responses or browse results (QS-2)

Rules run in this order: identify the caller; check the key; replay or conflict if the key was used; validate fields; check the category is approved; create the errand, lock the price and record history; find an eligible helper and create the assignment in the same transaction, or set backlog and queue the capacity notice; commit; dispatch notifications from the outbox. A notification failure never fails the request.

Status	Error code	Meaning	Errand created	Safe to retry
400	VALIDATION_FAILED	A field is missing, blank, too long or the deadline is in the past	No	After fixing the fields
401	NOT_AUTHENTICATED	The caller could not be identified	No	After signing in
403	ACCOUNT_NOT_CONFIRMED	The identity check reports the account cannot post	No	After the account is usable
409	IDEMPOTENCY_KEY_REUSED	The key was used with a different request body	No (original stands)	With a new key
422	CATEGORY_NOT_APPROVED	The category is not on the approved list	No	With an approved category
503	SERVICE_UNAVAILABLE	The database could not be reached	No	Yes, with the same key

Rationale. A new errand returns 201 Created; the same key and body returns 200 OK with the header Idempotent-Replay: true and an identical body. The idempotency key was chosen over duplicate detection by content because two real errands can look identical (two printing jobs); the accepted consequence is that the client generates and keeps a key. The assignment is created in the same call so the requester sees the true status immediately; the accepted consequence is that the call does more work.

Phase 2 added the endpoints POST /assignments/:id/accept and /decline, POST /errands/:id/confirm, GET /errands/:id (status, never contact details) and GET /errands/:id/contact (the gate), with the extra codes FORBIDDEN, NOT_FOUND, OFFER_EXPIRED, OFFER_NOT_ACTIVE, NOT_READY and CONTACT_NOT_AVAILABLE.

9.3 Deployment

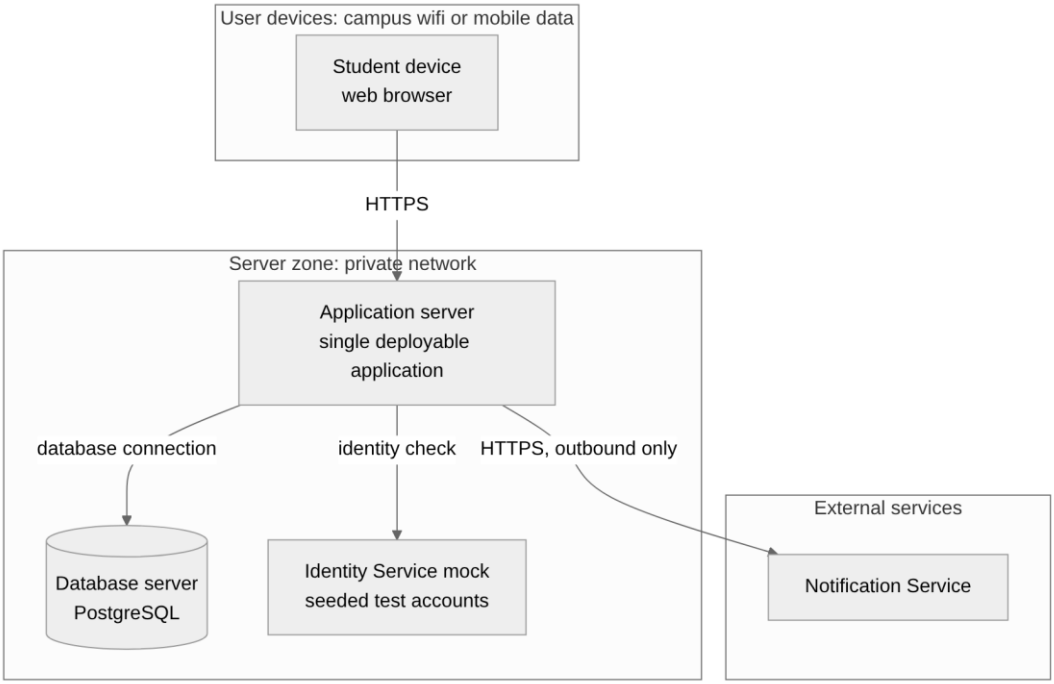


Figure 9.2: Deployment view with trust boundaries (source: models/deployment.drawio; this figure is a rendering of the same model).

A single deployable application runs on one application server and talks to one database server over a database connection and to the identity mock; it calls the Notification Service outbound over HTTPS only. The database accepts connections from the application node only. Contact and meeting details leave the server only through the confirmation gate.

9.4 Failure and recovery

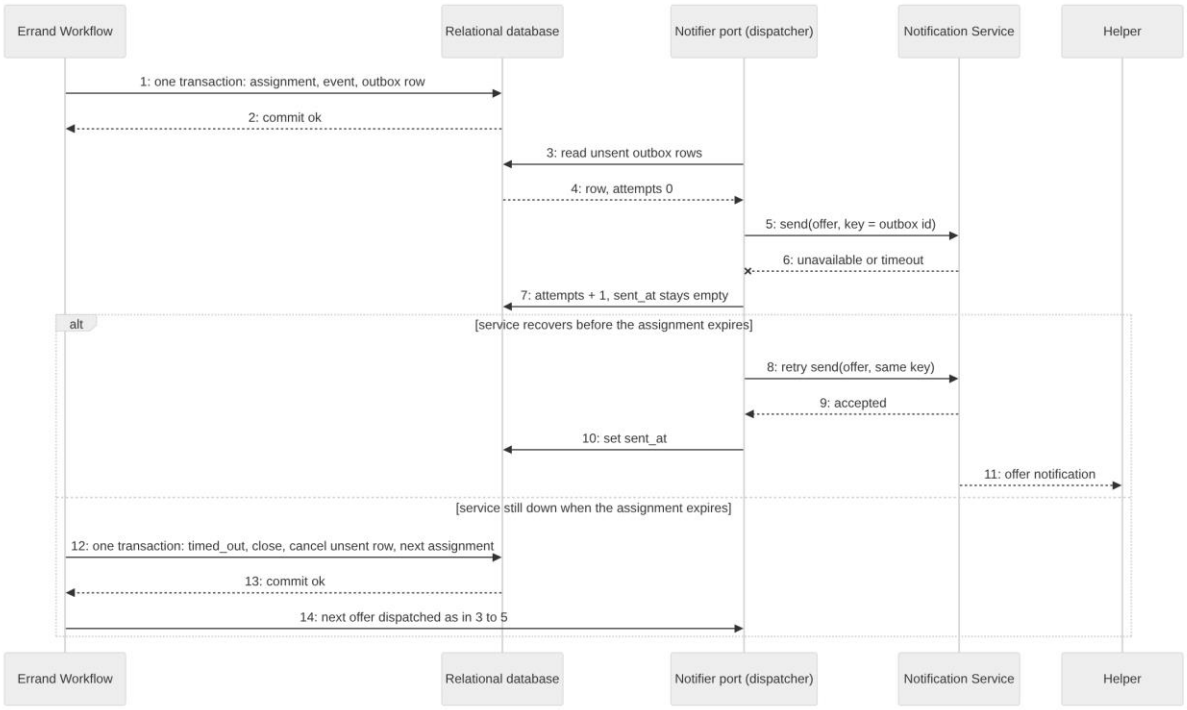


Figure 9.3: Notification Service unavailable when an assignment offer is sent (source: models/failure-recovery.drawio; this figure is a rendering of the same model).

Failure	Result	Recovery
Database unavailable during posting	Nothing stored; 503 SERVICE_UNAVAILABLE	Client retries with the same key (DI-7)
Crash between steps of posting	Transaction rolls back; no partial errand or assignment	Client retries with the same key

Failure	Result	Recovery
Notification Service unavailable when an offer is sent	Data already committed; the outbox row stays unsent	Dispatcher retries; if the assignment expires first it is closed and reassigned and the stale offer is cancelled
Application stopped while an assignment expires	Expiry is late, not lost	Stored expiry time; scheduler checks at start-up and at an interval
Client resubmits after a lost response	One errand only	Replay of the original response (DI-7)

10. Detailed design, patterns and framework

This section refines the architecture into implementable responsibilities and interfaces. Patterns are used only where they solve a named project problem; two were considered and rejected.

10.1 Classes and interfaces

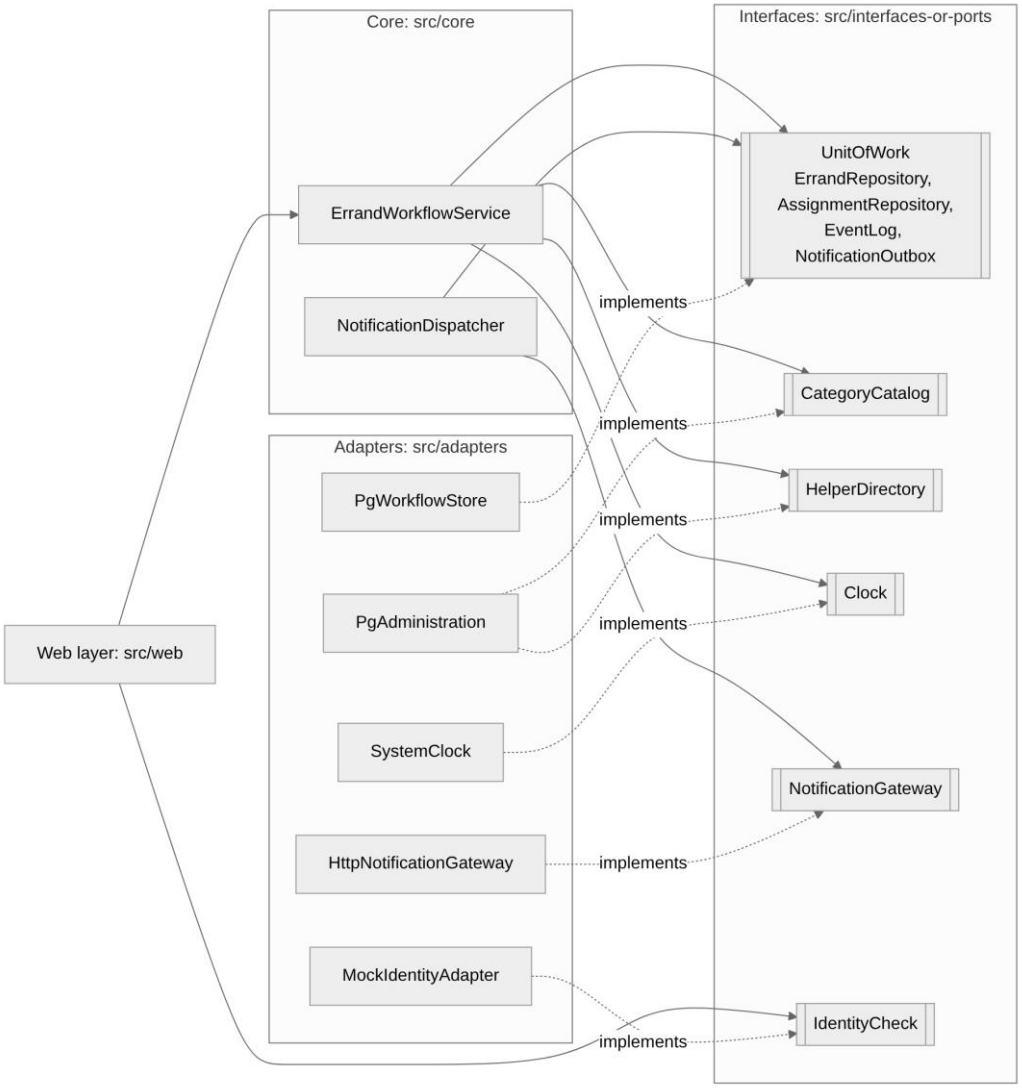


Figure 10.1: Dependency direction between the core, the interfaces and the adapters (source: models/detailed-class-design.drawio; this figure is a rendering of the same design, simplified; the built interface names are in Section 14.2).

The core (ErrandWorkflowService and NotificationDispatcher) imports only the interfaces in src/interfaces-or-ports. Adapters implement those interfaces and are named in one place, src/main.ts. The core never refers to an adapter, so the identity check, the notification service, the clock and the database can be replaced or faked without changing the workflow rules.

10.2 Pattern decisions

ID	Project problem	Pattern chosen	Consequence accepted	Evidence
PD-1	The identity check is mocked (D-007), the Notification Service can be down, and expiry depends on time; the workflow rules must not change when any of them changes	Ports and adapters (IdentityCheck, NotificationGateway, Clock)	Extra interfaces to keep stable; a port change touches core and adapter	Expiry tested with a fake clock; outage tested with a failing gateway
PD-2	Posting must create the errand, assignment, event and outbox row together, and a race must not give one helper two assignments (DI-1, DI-9, QS-5)	Repository with a unit of work	One more layer between service and tables; every new query needs a repository method	50 postings against 20 helpers: exactly 20 active assignments
PD-3	A notification outage must not fail the request or leave a stale offer (DI-11, DI-12)	Transactional outbox	Delivery is late, not immediate; the dispatcher must run; duplicates possible, so the outbox id is the idempotency key	Outage test: stale offer cancelled, only the live offer sent, once

10.3 Rejected alternatives

ID	Considered	Reason for rejection	Instead
PD-R1	Strategy pattern for helper selection (a HelperSelectionPolicy interface)	Only one selection rule exists; a second is in no requirement. An interface with one implementation adds a class, a binding and a test double (accidental complexity)	A plain function chooseHelpers inside the service; extract an interface if a second rule is ever required
PD-R2	Message broker with separate services for matching and notification	Not needed for 50 errands in 10 seconds; adds a component to deploy and a delivery guarantee to prove; already rejected in ADR-001 (Alternative B)	Outbox table in the existing database (PD-3)
PD-R3	Dependency-injection container and decorator-based framework	The container decides at run time which class fills an interface, hiding the wiring; the wiring for a handful of ports fits in one file	Hand wiring in src/main.ts; revisit if it outgrows one screen

10.4 Framework and dependency management (ADR-002)

Part	Choice	Reason
Language	TypeScript	Interfaces are checked by the compiler, so a breaking change to a port shows at build time
Runtime and web layer	Node.js with Express	Covers the Web/API layer only; no framework decides the structure of the core
Database	PostgreSQL	Partial unique indexes enforce DI-1 and DI-2; transactions cover DI-9
Database access	Plain SQL through one adapter	Keeps the transaction boundary visible; a full ORM would hide it
Tests	Vitest with fake adapters and a real test database	Runs the core with a fake clock and fake notification service

Dependency rules: packages are installed from a committed lock file; a new dependency needs a line in the pull request saying what it is for and what was considered instead; only adapters and the web layer import the database driver or HTTP client; dependencies are checked for known vulnerabilities before each phase submission.

Exit record. The interface with the greatest benefit is the unit of work with its four repositories, because it carries the guarantee behind the highest integrity risk (a helper getting two assignments). Its cost is that every new query becomes a repository method and the service tests need a fake store. The test that confirms it is 50 postings at once against 20 helpers.

11. User-interface, accessibility and usability

11.1 Users and tasks

Task	User	Scenario and success criteria
T1 Post an errand and get a confirmed helper	Requester (primary): a student short of time, on a phone, on campus wifi or mobile data that may drop	Posts a parcel collection before a Friday deadline. Success: a first-time user posts a valid errand in 90 seconds or less without help; can say the price and status from the screen; sees no contact detail before both confirm; a dropped connection does not create a second errand
T2 Respond to an errand offer within the window	Helper (secondary): a vetted student, possibly in a lecture or walking	Opens the offer notification and accepts or declines within 10 minutes. Success: answer in 2 actions from the notification; can tell the time left in words; an expired offer shows as expired, not as an error

Administrator screens (vetting, pricing, dispute resolution) are not designed in this phase. A realistic alternative was to design the dispute screen first; it was left out because it is used rarely by one person, while T1 and T2 decide whether an errand gets done.

11.2 Navigation model

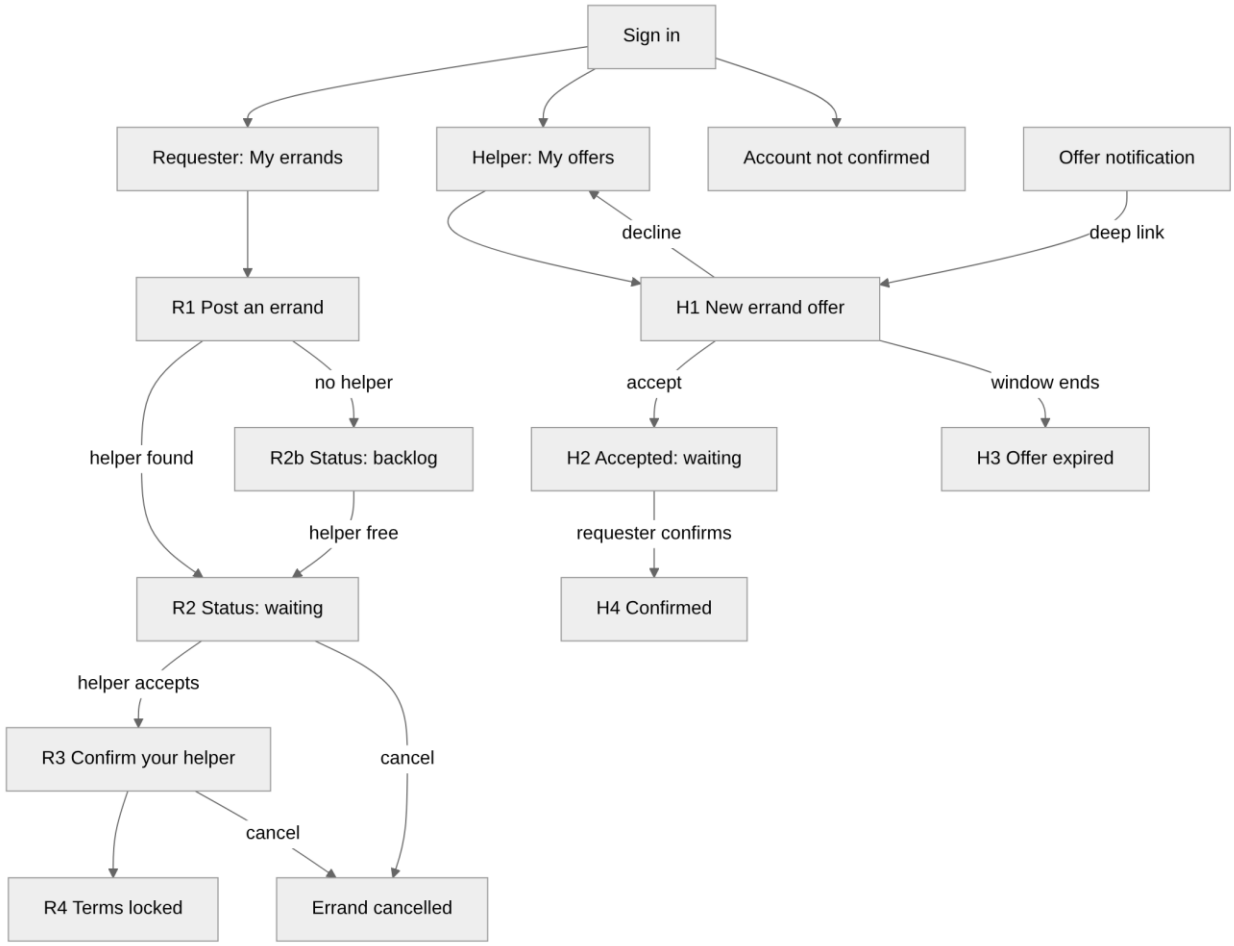


Figure 11.1: Navigation model (source: models/navigation.drawio; this figure is a rendering of the same model). Shortest paths: requester posts in 3 taps after sign in; helper responds in 2 actions from the notification.

11.3 Wireframes

Low-fidelity mobile wireframes for two roles are in prototype/wireframes.drawio (pages Requester and Helper), each screen annotated with the requirement or rule it serves.

Screen	Purpose	Key design points
R1 Post an errand	T1: capture category, details, deadline, meeting point	Persistent labels; price shown before posting and stated as fixed; meeting point hint says it is shown only after both confirm
R2 Status: waiting / R2b backlog	T1: show the true state	Status is a word in a box; expiry as clock time plus minutes left; backlog explains that nothing is needed from the user (QS-3); free cancel visible
R3 Confirm your helper / R4 Terms locked	T1: lock terms, then reveal contact	Terms listed before confirming; contact and meeting details appear only on R4 (QS-2)

Screen	Purpose	Key design points
H1 New errand offer	T2: decide quickly	Category, price, deadline, task and time left in words; Accept and Decline the same size; no meeting point or phone number
H2 Accepted: waiting / H3 Offer expired / H4 Confirmed	T2: show what happens next	Expired is a normal state with a clear message; contact details only on H4

11.4 Alternative states

Loading, empty, validation, error, unauthorised and unavailable states are designed for each screen in docs/ui-states.md and mapped to the API error codes of Section 9.2.

State	Post an errand (R1)	Helper offer (H1 to H4)
Loading	Post button shows Posting and is disabled; entered values stay; the key is kept so a second tap cannot create a second errand	Both buttons disabled while the answer is sent
Empty	Category list could not be loaded: message and Try again	No offers: a message that a notification will arrive
Validation	400: summary at the top and a message under each field; focus moves to the summary; values kept	Not applicable
Error	422 category no longer available: list refreshes; 503 or offline: nothing was saved, try again, values kept	OFFER_EXPIRED: shown as the Offer expired screen
Unauthorised	401 sign in; 403 ACCOUNT_NOT_CONFIRMED: confirm your university email	A requester opening a helper page: this page is for helpers
Unavailable or stale	Offline: the errand has not been posted	If the offer expires while offline it is passed on; stale status shows last-updated time and Refresh

11.5 Accessibility

docs/accessibility-checklist.md lists 12 checks based on WCAG 2.2 level AA: keyboard access, visible focus, persistent labels (A-3), status never by colour alone (A-4), error summary with focus (A-5), live-region announcements limited to once a minute for the countdown (A-6), contrast, 360 px layout and 200 percent text, 44 px touch targets, no hidden Cancel or Decline, headings and titles. Five checks are marked Designed in the wireframes (A-3, A-4, A-5, A-10, A-11); the others are marked To verify and need the built interface. Two design choices are recorded with their alternatives: the time left is shown as a clock time plus words updated once a minute (rejected: a ticking countdown, hard to read and noisy for screen readers), and status is a word in a box (rejected: a coloured chip, which fails for colour-blind users).

11.6 Usability evaluation

The test plan (three students outside the team, task-based, think-aloud, with success criteria) and a team walkthrough are in evidence/usability-test.md. The walkthrough led to change C-1: the helper's time left is given as a clock time plus words ('Respond by 14:10 (9 min left)') instead of only a countdown. The three-person test itself has not yet been run; its observations and any further change are to be recorded in that file. This is listed as a limitation in Section 15.

12. Implementation vertical slice

12.1 What is implemented

The approved core use case, Post Errand Request, runs end to end through the API, the application logic and PostgreSQL, and continues through helper acceptance and requester confirmation:

- A requester posts an errand with an idempotency key; the category is checked, the standard price is locked, a helper is assigned, history is recorded and an offer is queued, all in one transaction.
- If no helper is free the errand is flagged backlog and the administrator is notified; backlog errands are retried when a helper becomes free.
- The helper accepts or declines; a decline or a 10-minute timeout offers the errand to the next helper, and a stale unsent offer is cancelled.
- The requester confirms; only then can either party read the meeting point and phone numbers (the confirmation gate).

12.2 Repository structure

Path	Contents
src/interfaces-or-ports/	Interfaces the core depends on (ADR-002)
src/core/	Errand Workflow rules: errand-workflow-service.ts, notification-dispatcher.ts, errors.ts
src/adapters/	PostgreSQL store, administration data, mock identity, clock, notification gateway
src/web/, src/main.ts	HTTP routes and error mapping; the only file that names concrete adapters
migrations-or-schema/	001_init.sql (tables and integrity rules), 002_sample_data.sql (synthetic data)
tests/	Automated tests and results summary
docs/, models/, decisions/, design-record/, prototype/, evidence/	Requirements, models, ADRs, decisions, wireframes and evidence

12.3 Setup and run

Requirements: Node.js 20 or newer and PostgreSQL 14 or newer. From the repository root: npm install; create two empty databases; set DATABASE_URL; npm run db:setup -- --sample (creates the tables and loads synthetic sample data); npm start (listens on port 3000). Sample accounts are bearer tokens of the mock identity (requester-1, helper-1 and so on). The README gives the exact commands, a curl walk-through of the slice, and the test command; docs/demo-script.md is the five-minute demonstration.

12.4 What is not built

There is no user interface (the Section 11 wireframes are designs only); cancel, work-in-progress, completion, no-show and dispute flows and the administrator functions are not built; identity is mocked (D-007). Section 14 lists every difference.

13. Testing and results

Automated tests (tests/workflow.test.ts, 21 tests) run against a real PostgreSQL database with a fake clock and a fake notification service, and check outcomes and side effects in the database, not only that code runs. Results of the recorded run (30 October 2026, Node.js 22, PostgreSQL 16): 21 passed, 0 failed; the type check reports no errors. The summary is in tests/results-summary.md.

Type	Test	Outcome checked	Traces to
Success	Post an errand	201, AWAITING_ACCEPTANCE, expiry exactly 10 minutes later; one errand, one active assignment, two history events, one offer queued; meeting point absent from the response	FR-01, FR-02, QS-1, QS-2
Success	Price lock	Stored price stays 20 after the category price changes to 30; the next errand costs 30	BR-D2, DI-6
Success	Backlog	No free helper: POSTED with backlog, administrator flag queued, no assignment	QS-3
Success	Backlog retry	A helper who becomes free is offered the oldest waiting errand	QS-3
Success	Accept, confirm, reveal	Contact details refused before both confirm, shown to both after, refused to anyone else	FR-05, QS-2, DI-8
Rule or validation	Bad fields, missing key	400 naming the fields; nothing created	FR-01
Rule or validation	Non-approved or unknown category	422; no errand and no history row	FR-13, BR-D1
Rule or validation	Wrong or unconfirmed caller	401, 403 ACCOUNT_NOT_CONFIRMED, 403 FORBIDDEN; nothing created	D-007
Rule or validation	Duplicate submission	Same key and body replays; different body gives 409; five at once create one errand	DI-7
Rule or validation	Another helper answers an offer; late accept	404; 409 OFFER_EXPIRED; nothing changed	FR-03, QS-1
Rule or validation	Decline and timeout	Next helper offered; outcome recorded; unsent offer cancelled; nothing at 9 minutes, reassigned at 10	FR-03, FR-04, QS-1, DI-11
Failure	Notification Service down	Posting still succeeds; offer unsent with attempt counted; stale offer cancelled on expiry; after recovery only the live offer is sent, once	DI-11, DI-12

Type	Test	Outcome checked	Traces to
Failure	Database unreachable	503 SERVICE_UNAVAILABLE; nothing stored	core-operation.md
Concurrency	50 postings, 20 helpers	Exactly 20 active assignments, none doubled, 30 in backlog	DI-1, QS-5
Database rules	Direct changes	Outcome cannot change; history cannot be edited or deleted; a second active assignment for one errand is refused	DI-2, DI-3, DI-10

13.1 Quality evidence

Scenario	Evidence	Limit of the evidence
QS-1	Timeout, late-accept and outage tests with a fake clock	The 30-second timer and the 1-minute measure were not measured end to end
QS-2	Gate test: no contact detail in any response before both confirm	Verified for the API only; no interface exists
QS-3	Backlog and backlog-retry tests	None significant for the built path
QS-4	History written with each built transition; append-only rule tested	Only the built transitions; no dispute review
QS-5	50 against 20 test: zero doubled, 30 in backlog	Correctness proven; the 10-second time was not measured; post-time order is not guaranteed under concurrency

13.2 Not covered by automated tests

The user interface (none is built); the 30-second timer itself; the real Notification Service (only a fake was used); cancel, in-progress, completion, dispute and administrator flows (not built); load above 50 postings; running more than one application instance.

14. Reconciliation and change history

This section compares the design with the code and tests as they stand on 30 October 2026. Differences are disclosed; the diagrams were not edited to hide them. The full matrix is docs/reconciliation-matrix.md.

14.1 Requirements to code to tests

Requirement	Code	Test	Result and difference
FR-01 Post errand	postErrand, POST /errands	Normal; validation; duplicate	Done at API level; no user interface
FR-02 Automatic assignment	offerToNextHelper	Normal; 50 against 20	Done; helpers tried in id order (TD-7)
FR-03 Accept or decline	helperAccept, helperDecline	Decline; late accept; wrong helper	Done
FR-04 Reassign on decline or timeout	closeAndReassign, expireDueAssignments	Decline; timeout	Done
FR-05 Lock terms, reveal contact	requesterConfirm, contactFor	Confirmation gate	Differs: terms are locked by the requester's confirmation flag, not a separate status
FR-06 to FR-10 Cancel, no-show, dispute, resolution	dispute table only	Database rules only	Not built
FR-11, FR-12 Register helpers, set price	none	none	Not built; helpers and prices come from sample data
FR-13 Reject non-approved category	postErrand step 5	Non-approved; unknown	Done
FR-14 No prohibited tasks	none	none	Not built; depends on the category list
FR-15 No payment data	no payment fields in schema	inspection	Done by omission
FR-16 Account by email link	MockIdentityAdapter	Authorisation	Differs (D-007): mocked fixed tokens; FR text and D-007 still disagree
FR-17 Notifications	outbox, dispatcher	Outage	Partial: real service not integrated

14.2 Models to code

Artefact	Result and difference
Sequence (Figure 7.2)	Agrees on the order: post, assign, helper accepts, requester confirms, lock, reveal. The step 6 acknowledgement is the API response
Lifecycle (Figure 7.1)	Partial: built transitions are POSTED to AWAITING_ACCEPTANCE, AWAITING_ACCEPTANCE to ACCEPTED and back to POSTED; the other states are allowed values but nothing moves an errand into them
Component view (Figure 8.1)	Partial: Web, Errand Workflow and Administration exist; Dispute is not built; the workflow adapter reads the requester table to copy the phone number because requester ownership was never decided
Logical data model (Figure 9.1)	Agrees with all tables and rules; notification_outbox gained assignment_id and recipient_role; prices are whole numbers
API contract (Section 9.2)	Agrees on fields, steps and the six codes; six codes and five endpoints added; the category is read just before the transaction, not inside it
Failure recovery (Figure 9.3)	Agrees in effect and is tested; no next-retry time or back-off: the dispatcher retries on every 30-second pass
Class design (Figure 10.1)	Structure holds (the core imports only interfaces). Names changed: CategoryCatalog.isApproved and standardPrice became approvedCategoryPrice; HelperDirectory.eligibleHelpers became availableHelpers; AssignmentRepository.save became tryCreateActive and close; NotificationOutbox.nextUnsent became listUnsent and markFailed; the confirmation gate is the method contactFor; the expiry scheduler is a timer in main.ts
Wireframes (Section 11)	Not built; API error codes match docs/ui-states.md so the screens can be built against it

14.3 Artefacts updated and change history

Change	Why	Where
Lifecycle: added the edge POSTED to CANCELLED	FR-06: free cancellation must cover backlogged errands, which are POSTED	Figure 7.1; models/state/errand-lifecycle.drawio
Sequence: opt frame for rejection, backlog branch, requester confirmation, lockTerms gated on both confirmations	The sequence contradicted QS-2 and left the backlog and rejection paths out	Figure 7.2; models/sequence/sequence-core-use-case.drawio
QS-1 and QS-5 measures tightened	The measures were not testable as written	Section 5
Traceability matrix: Pending rows replaced by acceptance criteria and checks	Rows were marked pending	Section 6.4, Appendix B
Use case added: Decline Assignment; consistency matrix across artefacts	FR-03 and FR-04 had no use case	docs/use-cases, docs/consistency-matrix.md
Backlog recorded as a flag on a POSTED errand, not a status	Keeps the lifecycle diagram unchanged	Section 9.1
ErrandStatus in the interface definitions aligned with the lifecycle	Early drafts used status names not in the diagram	src/interfaces-or-ports/types.ts

Open items carried forward: FR-16 and D-007 disagree on email confirmation; AC-6 to AC-12 and AC-17 are defined in the traceability matrix but not yet written in docs/acceptance-criteria.md; the requester table has no owning module; whether a helper may withdraw after confirming is undecided.

15. Risks, technical debt and release recommendation

15.1 Technical debt

ID	Debt	Impact	Suggested action
TD-1	No user interface; wireframes only	High: students cannot use the product	Build R1, R2, H1, H2 first
TD-2	No work after acceptance (start, complete, cancel, no-show, dispute); an accepted errand never ends, so its helper stays busy	High: capacity leak	Add complete and cancel first

ID	Debt	Impact	Suggested action
TD-3	Identity is mocked with fixed tokens (D-007); FR-16 says email-link confirmation	High outside a demo	Replace the adapter behind IdentityCheck; reconcile FR-16 with D-007
TD-4	Administrator functions are SQL only	Medium	Build after TD-2
TD-5	Notification retries have no back-off or limit; real service never run	Medium	Add next-retry time and a limit; test against the real service
TD-6	Expiry and backlog timer in-process; more than one instance untested	Medium	Test two instances before scaling
TD-7	Helper choice by lowest id; concurrent postings not in post-time order (QS-5 wording)	Medium	Decide the fairness rule
TD-8	One integration suite; no unit tests with fakes, no UI or accessibility tests, no load test above 50	Medium	Add unit tests and a timer test
TD-9	Category and price read just before the transaction, not inside it	Low	Read inside the transaction or update the contract
TD-10	Documents and code disagree in places (FR-16, AC-6 to AC-12, requester ownership)	Medium	Update after review
TD-11	No rate limiting, HTTPS set-up, secret handling or monitoring	High for real use	Add before any real users

15.2 Residual risks

ID	Risk	Effect	Mitigation now
RR-1	Contact details leak through a future endpoint	QS-2 broken	Only contactFor reads them; add a test for every new response shape
RR-2	A helper who never completes holds capacity forever	Errands stay in backlog while helpers are idle	No real pilot until complete or cancel exists
RR-3	Real students use the mock login	Impersonation	Synthetic data only
RR-4	Notification outage longer than the acceptance window	Helpers miss offers; requesters wait	Retry limit and administrator alert (TD-5)
RR-5	Behaviour under real load unknown	QS-5 time measure unproven	Measure the 10-second target before claiming it
RR-6	Dispute evidence incomplete for later transitions	QS-4 only partly met	Record events as each transition is added
RR-7	The usability test with three users has not been run	Design changes are from a walkthrough, not observed difficulty	Run the test and record it in evidence/usability-test.md

15.3 Release recommendation

Not ready for production or for real student use. Recommended: a conditional release as a supervised demonstration or classroom pilot using synthetic data only, on these conditions: (1) the mock identity stays and no real student data is entered; (2) complete and cancel exist before any test with more than one errand per helper; (3) the user interface for the post and respond paths exists and passes the accessibility checklist; (4) the notification retry limit and a real-service test are done; (5) HTTPS, secret handling and rate limiting are in place; (6) the QS-5 10-second target is measured and the fairness rule decided. The evidence for what is claimed is the passing automated tests, the schema rules, and the reconciliation in Section 14. Nothing here claims production readiness.

16. Academic integrity and AI-use record

Claude was used as a supplementary tool to assist with drafting and reviewing selected laboratory artefacts, code, and documentation. All outputs were reviewed, edited, and validated by the team before being committed to the shared repository. Figures in Sections 7–11 were generated using the same models, while the editable source files are included in the repository.

Verification: the team reviewed the artefacts before committing them to the shared repository through Git, and the commit history records what was committed. The vertical slice is verified by 21 automated tests and a type check (Section 13), not by the assistant's statement that it works. Differences between design and code were found by comparing the artefacts and are disclosed in Section 14. Limits: the usability test with users, the real Notification Service and the user interface have not been verified because they do not yet exist or have not been run (Section 15).

References

Bass, L., Clements, P. and Kazman, R. (2021) Software Architecture in Practice. 4th edn. Boston: Addison-Wesley.

Cockburn, A. (2005) Hexagonal architecture (ports and adapters). Available at: https://alistair.cockburn.us/hexagonal-architecture/ (Accessed: 30 October 2026).

Fowler, M. (2002) Patterns of Enterprise Application Architecture. Boston: Addison-Wesley. (Repository; Unit of Work.)

ISO/IEC (2023) ISO/IEC 25010:2023 Systems and software engineering - Product quality model. Geneva: ISO.

Nygard, M. (2011) Documenting architecture decisions. Available at: https://www.cognitect.com/blog/2011/11/15/documenting-architecture-decisions (Accessed: 30 October 2026).

Richardson, C. (2018) Microservices Patterns. Shelter Island: Manning. (Transactional outbox.)

W3C (2023) Web Content Accessibility Guidelines (WCAG) 2.2. Available at: https://www.w3.org/TR/WCAG22/ (Accessed: 30 October 2026).

Appendices

Appendix A: Approval form

The project was approved with minor conditions (R1-R6). The approval form and the lecturer's feedback are in the repository: docs/project-proposal-and-approval.docx and docs/lecturer-feedback.md. Each condition is traced to a design record in Section 2.3.

Appendix B: Traceability matrix (Phase 2)

The matrix in Section 6.4 is the same table. It is repeated in the repository as docs/traceability-matrix.md. Quality and interface traceability added in Phase 2:

Task or screen	Requirement	Quality scenario or rule
T1, R1	FR-01, FR-13	BR-D1, BR-D2
T1, R2	FR-02, FR-06	QS-1 (expiry shown)
T1, R2b	FR-02	QS-3 (backlog explained)
T1, R3, R4	FR-05	QS-2
T2, H1	FR-03, FR-17	QS-1
T2, H3	FR-04	QS-1
T2, H2, H4	FR-05	QS-2

Appendix C: Architecture decision records

ADR	Decision	Alternatives considered	Status
ADR-001	Modular application: one deployable application with four parts, one relational database and ports for identity and notifications	B: separate services with a message queue (rejected: coordination cost, split history, build and run cost)	Accepted for Phase 2 implementation
ADR-002	Ports and adapters, repositories in a unit of work, transactional outbox; TypeScript, Node.js and PostgreSQL wired by hand	Direct calls from core to database and service; broker with separate services; strategy interface for selection; dependency-injection container	Accepted

Full text: decisions/ADR-001-architecture.md and decisions/ADR-002.md.

Appendix D: Test log

Command npm test, recorded run on 30 October 2026 (Node.js 22, PostgreSQL 16): 21 tests passed. The full list with the outcome each test checks is in tests/results-summary.md and tests/workflow.test.ts.